

METODOLOGÍA DE LA PROGRAMACIÓN

TEMA 2: DISEÑO FORMAL DE ALGORITMOS ITERATIVOS



Objetivos

- ✓ Conocer la semántica axiomática de un lenguaje y como especificarla
- ✓ Formalizar la semántica intuitiva de las instrucciones iterativas
- ✓ Aprender a derivar formalmente programas iterativos sencillos
- ✓ Aprender a verificar a posteriori programas derivados

Índice: Tema 2

2.1 El sistema formal de Hoare

2.1.1 Introducción

2.1.2 Reglas generales

2.1.3 Reglas de las instrucciones del lenguaje

2.1.4 Instrucciones iterativas

2.2 Derivación formal de algoritmos iterativos

2.2.1 Derivación de instrucciones de asignación

2.2.2 Derivación de bucles

2.3 Verificación formal de algoritmos iterativos

2.3.1 Proceso de verificación

Índice: Tema 2

2.1 El sistema formal de Hoare

2.1.1 Introducción

2.1.2 Reglas generales

2.1.3 Reglas de las instrucciones del lenguaje

2.1.4 Instrucciones iterativas

2.2 Derivación formal de algoritmos iterativos

2.2.1 Derivación de instrucciones de asignación

2.2.2 Derivación de bucles

2.3 Verificación formal de algoritmos iterativos

2.3.1 Proceso de verificación

- Los programas expresados mediante **lenguajes imperativos** se caracterizan por estar contruidos mediante una secuencia de ordenes o instrucciones.
- Así, un programa del tipo $\{Q\} S \{R\}$ puede verse como una composición secuencial de instrucciones individuales (s_i) satisfaciendo una especificación dada, con sus correspondientes asertos (A_i):
 - ✓ $\{Q \equiv A_0\} s_1 \{A_1\} s_2 \{A_2\} s_3 \cdots s_n \{A_n \equiv R\}$
- La **semántica (axiomática)** de un lenguaje es el conjunto de reglas que permiten conocer si cada instrucción satisface su especificación. Permiten:
 - ✓ Dados Q, S, R deducir si se satisface $\{Q\} S \{R\}$.
 - ✓ Dados S y R deducir Q .
- Se representan mediante “*numerador / denominador*” y se interpretan como “*de la veracidad del numerador se infiere la veracidad del denominador*”.

- Un conjunto de tales reglas es el sistema formal de **Hoare**, un sistema de reglas de inferencia que permiten razonar sobre las instrucciones de un algoritmo, bien sea para calcularlas o para verificar su corrección.
- Estas reglas se pueden clasificar en tres grupos:
 - ✓ Reglas generales.
 - ✓ Reglas de las instrucciones del lenguaje.
 - ✓ Instrucciones iterativas.
- Denotaremos por **$Dom(E)$** al predicado que defina a todos los valores del dominio de las variables libres de **E** para los que sea posible evaluar **E** .
 - ✓ La expresión $\{E \equiv x + y > 0\}$ es evaluable para cualquier valor de x e y , siendo $\{Dom(E) \equiv Cierto\}$.
 - ✓ La expresión $\{E \equiv x/y\}$ no es evaluable para $y=0$, siendo $\{Dom(E) \equiv y \neq 0\}$.

Índice: Tema 2

2.1 El sistema formal de Hoare

2.1.1 Introducción

2.1.2 Reglas generales

2.1.3 Reglas de las instrucciones del lenguaje

2.1.4 Instrucciones iterativas

2.2 Derivación formal de algoritmos iterativos

2.2.1 Derivación de instrucciones de asignación

2.2.2 Derivación de bucles

2.3 Verificación formal de algoritmos iterativos

2.3.1 Proceso de verificación

- Reglas de reforzamiento de la precondition y debilitamiento de la postcondición (o reglas de la consecuencia).

- ✓ Sirven para introducir asertos más fuertes/débiles.

- ✓ La primera de estas reglas dice que siempre se puede fortalecer la precondition:

$$\frac{\{Q\}S\{R\}, Q' \Rightarrow Q}{\{Q'\}S\{R\}}$$

- ✓ La segunda dice que siempre se puede debilitar la postcondición:

$$\frac{\{Q\}S\{R\}, R \Rightarrow R'}{\{Q\}S\{R'\}}$$

- Reglas de composición de precondiciones y postcondiciones.

- ✓ Esta regla dice que siempre se pueden componer las precondiciones y las postcondiciones si se hacen ambas con el mismo operador ($\wedge$ ó $\vee$):

$$\frac{\{Q_1\}S\{R_1\}, \{Q_2\}S\{R_2\}}{\{Q_1 \wedge Q_2\}S\{R_1 \wedge R_2\}, \{Q_1 \vee Q_2\}S\{R_1 \vee R_2\}}$$

Índice: Tema 2

2.1 El sistema formal de Hoare

2.1.1 Introducción

2.1.2 Reglas generales

2.1.3 Reglas de las instrucciones del lenguaje

2.1.4 Instrucciones iterativas

2.2 Derivación formal de algoritmos iterativos

2.2.1 Derivación de instrucciones de asignación

2.2.2 Derivación de bucles

2.3 Verificación formal de algoritmos iterativos

2.3.1 Proceso de verificación

➤ Instrucciones “nada” y “abortar”:

- ✓ El axioma que define la **instrucción nada** es no realizar acción alguna. La instrucción siempre termina, y su efecto sobre el estado del cómputo es nulo.

$$\overline{\{Q\} \text{ nada } \{Q\}}$$

- ✓ Esta regla equivale a:
$$\frac{Q \Rightarrow R}{\{Q\} \text{ nada } \{R\}}$$

- ✓ La **instrucción abortar**, o bien interrumpe bruscamente, o bien prolonga indefinidamente el bien prolonga indefinidamente el cómputo, impidiendo que se alcance cualquier estado útil. No existe ningún estado de partida que garantice que la instrucción abortar termina en un estado definido.

$$\overline{\{falso\} \text{ abortar } \{Q\}}$$

➤ Instrucción de asignación:

- ✓ Sea la **instrucción de asignación** $x := E$, donde x es una variable y E una expresión del mismo tipo que x . La regla permite calcular una precondition admissible a partir de la postcondition deseada para la instrucción.

$$\overline{\{Dom(E) \wedge R_x^E\} x := E \{R\}}$$

- ✓ El predicado R_x^E se construye sustituyendo en R todas las apariciones libres de x por la expresión E .
- ✓ Dicha precondition incluye sólo estados en los que E es evaluable, ya que de otro modo la instrucción abortaría.
- ✓ Así, la expresión E debe satisfacer antes de la ejecución de la instrucción las mismas propiedades que x satisface tras su ejecución.

➤ Instrucción de asignación (continuación...):

- ✓ Esta regla proporciona la forma de obtener la precondition más débil de un segmento de código ***C***, ***pmd(C,R)***, que garantiza que su ejecución de lleve a un estado que satisfaga ***R***.

- ✓ **Ejemplo:** Los siguientes son ejemplos de precondiciones más débiles.

$$\{x + y = 10\}x := x + y\{x = 10\}$$

$$\{\exists j.(x = 2j)\}z := x\{\exists j.(z = 2j)\}$$

- ✓ Además, de esta regla y de la de reforzamiento de la precondition se deduce la siguiente regla, que da la forma de demostrar la corrección de ***{Q} C {R}***, que consiste en demostrar la corrección de ***Q ⇒ pmd(C,R)***.

$$\frac{Q \Rightarrow R_x^t}{\{Q\}x := t\{R\}}$$

➤ Instrucción de asignación (continuación...):

- ✓ Cuando aparezcan en la expresión E componentes de vectores o, en general, cualquier tipo de operaciones parciales, se hará constar expresamente en la precondition el dominio de E .

- ✓ Así una precondition Q para la especificación $\{Q\}x := a[i]\{x > 0\}$ es:

$$Q \equiv Dom(E) \wedge R_x^E \equiv 1 \leq i \leq n \wedge a[i] > 0$$

- ✓ Admitiendo en el lenguaje asignaciones múltiples $\langle x_1, \dots, x_n \rangle := \langle E_1, \dots, E_n \rangle$ la regla de asignación generalizada sería:

$$\frac{}{\{R_{x_1, \dots, x_n}^{E_1, \dots, E_n} \langle x_1, \dots, x_n \rangle := \langle E_1, \dots, E_n \rangle \{R\}}$$

➤ Instrucción de asignación (continuación...):

✓ Determinar Q para las siguientes R en la asignación $\{Q\} x := x + 1 \{R\}$

1. $R \equiv x = 7$

2. $R \equiv x + y > 0$

3. $R \equiv y = 2^k$

4. $R \equiv \exists x \geq 0. y = 2^x$

➤ Instrucción de composición de instrucciones:

- ✓ La **composición** secuencial de dos instrucciones S_1 y S_2 , denotada $S_1; S_2$, se utiliza para encadenar cálculos.
- ✓ Así, esta regla sirve para razonar con secuencias de instrucciones:

$$\frac{\{Q\}S_1\{P\}, \{P\}S_2\{R\}}{\{Q\}S_1; S_2\{R\}}$$

- ✓ La regla dice que dos instrucciones S_1 y S_2 , en este orden, satisfacen una especificación con precondition Q y postcondición R , si existe un aserto intermedio P , que sea postcondición de la primera y precondition de la segunda.
- ✓ Aparentemente es necesario inventar un predicado P que describa el conjunto de estados alcanzados después de ejecutar S_1 y antes de ejecutar S_2 , y demostrar que se satisfacen cada instrucción por separado.

➤ Instrucción de composición de instrucciones (continuación...):

- ✓ Calcular la precondition Q más general del siguiente programa:

$$\{Q\}$$

$$x := x + y$$

$$y := x - y$$

$$x := x - y$$

$$\{R\}$$

➤ Instrucción de selección:

- ✓ La forma más simple de una **instrucción de selección** es:

Si B entonces S_1 si_no S_2 fsi

- ✓ Su ejecución comienza evaluando la condición booleana B en el estado en curso. Si resulta ser cierta, se ejecuta S_1 , en caso contrario se ejecuta S_2 (si existe).
- ✓ Esta regla dice que para demostrar que la instrucción anterior satisface una especificación con precondition Q y postcondición R , habrá que demostrar que son correctas las dos especificaciones siguientes:

$\{Q \wedge B\}S_1\{R\}$ y $\{Q \wedge \neg B\}S_2\{R\}$

- ✓ La regla en cuestión es:

$$\frac{\{Q \wedge B\}S_1\{R\}, \{Q \wedge \neg B\}S_2\{R\}}{\{Q \wedge \text{Dom}(B)\}\text{Si } B \text{ entonces } S_1 \text{ si_no } S_2 \text{ fsi}\{R\}}$$

➤ Instrucción de selección (continuación...):

- ✓ Y particularizada para una instrucción de selección con una rama vacía:

$$\frac{\{Q \wedge B\}S\{R\}, Q \wedge \neg B \Rightarrow R}{\{Q \wedge Dom(B)\}\text{Si } B \text{ entonces } S \text{ fsi}\{R\}}$$

- ✓ La regla obliga a que el programador conjeture un predicado Q que sirva como precondition de la instrucción, y habrá que demostrar que se satisfacen las dos especificaciones del antecedente de la regla.
- ✓ Como con la asignación, se puede partir de la postcondición R , e ir calculando hacia atrás las condiciones Q_1 y Q_2 correspondientes a S_1 y S_2 , y construir la precondition de la instrucción condicional mediante la siguiente regla alternativa:

$$\frac{\{Q_1\}S_1\{R\}, \{Q_2\}S_2\{R\}}{\{Dom(B) \wedge ((Q_1 \wedge B) \vee (Q_2 \wedge \neg B))\}\text{si } B \text{ entonces } S_1 \text{ si no } S_2 \text{ fsi}\{R\}}$$

➤ Instrucción de selección (continuación...):

- ✓ La segunda forma de instrucción condicional es la **instrucción caso**:

$$\frac{\{Q \wedge B_1\}S_1\{R\}, \dots, \{Q \wedge B_n\}S_n\{R\}}{\{Q \wedge UNA(B_1, \dots, B_n)\} \mathbf{caso} \ B_1 \rightarrow S_1[\dots [] B_n \rightarrow S_n \mathbf{fcaso} \{R\}}$$

siendo $UNA(B_1, \dots, B_n) \equiv \left(\bigwedge_{i=1}^n Dom(B_i) \right) \wedge (N_i \in \{1..n\}.B_i) = 1$

- ✓ Donde las B_i son expresiones booleanas y las S_i secuencias de instrucciones. Se exige que, en cada ejecución, exactamente una de las condiciones B_i sea cierta. Si ninguna o más de una lo es, la instrucción aborta.

- ✓ Alternativa:

$$\frac{\{Q_1\}S_1\{R\}, \dots, \{Q_n\}S_n\{R\}}{\{Q \wedge UNA(B_1, \dots, B_n)\} \mathbf{caso} \ B_1 \rightarrow S_1[\dots [] B_n \rightarrow S_n \mathbf{fcaso} \{R\}}$$

siendo $Q \equiv (Q_1 \wedge B_1) \vee \dots \vee (Q_n \wedge B_n)$

➤ Instrucción de selección (continuación...):

- ✓ Calcular una precondition Q para el siguiente programa:

$\{Q\}$

si $x < 0$ *entonces*

$x := x + 1$

sino

$x := x - 1$

fsi

$\{R \equiv x \geq 0\}$

Índice: Tema 2

2.1 El sistema formal de Hoare

2.1.1 Introducción

2.1.2 Reglas generales

2.1.3 Reglas de las instrucciones del lenguaje

2.1.4 Instrucciones iterativas

2.2 Derivación formal de algoritmos iterativos

2.2.1 Derivación de instrucciones de asignación

2.2.2 Derivación de bucles

2.3 Verificación formal de algoritmos iterativos

2.3.1 Proceso de verificación

- La forma básica de **iteración** es la instrucción **mientras**, representando un bucle que puede iterar cero o más veces, cuya sintaxis es:

mientras B hacer S fmientras

- El bucle comienza evaluando la condición booleana **B** :
 - ✓ Si ésta es falsa, el bucle es equivalente a la instrucción **nada**.
 - ✓ En caso contrario, se ejecuta el cuerpo **S** de la iteración y se vuelve a evaluar la condición **B** .
 - ✓ Este proceso se repite hasta que la condición **B** se hace falsa (de suceder ello alguna vez).
- Para esta instrucción no es práctico dar una regla que permita calcular la precondition a partir de la postcondición.
- En su lugar, el programador conjetura un predicado especial que llamaremos **invariante** sin el cual no es posible razonar sobre el efecto del bucle.

➤ **Ejemplo:** Explicación intuitiva del concepto de **invariante**.

✓ Si por ejemplo, se tiene el siguiente segmento de código:

```
i := 0; q := 0; p := 1
mientras  i < n  hacer
    i := i + 1;  q := q + p;  p := p + 2
fmientras
```

✓ Si se listan los estados del algoritmo antes de cada iteración se tendrían los valores de la siguiente tabla:

estado	i	q	p
P_0	0	0	1
P_1	1	1	3
P_2	2	4	5
$\vdots$	$\vdots$	$\vdots$	$\vdots$

Donde se ve que las variables cumplen la relación expresada por el predicado:

$$q = i^2 \wedge p = 2 * i + 1$$

Este predicado sería el invariante del bucle.

- Se llama **invariante** de un bucle a un aserto ***P*** que describe todos los estados por los que atraviesan las variables del bucle, observados justo antes de evaluar la condición ***B*** de terminación del bucle. Por tanto:
 - ✓ El invariante se tiene que cumplir antes y después de cada iteración y, en particular, antes de la primera y después de la última, siendo a la vez, por tanto, precondition y postcondición del cuerpo ***S*** de la iteración: $\{P \wedge B\}S\{P\}$
 - ✓ Además, se debe verificar que $P \Rightarrow \text{Dom}(B)$, ya que de lo contrario habría estados en que ***B*** no sería evaluable, abortándose la ejecución del algoritmo.
 - ✓ Si el bucle termina, lo hace satisfaciendo el invariante ***P*** y la negación de la condición ***B***, por lo que la postcondición del bucle será: $P \wedge \neg B$
 - ✓ Con todo ello:
$$\frac{P \Rightarrow \text{Dom}(B), \{P \wedge B\}S\{P\}}{\{P\}\text{mientras } B \text{ hacer } S \text{ fmientras } \{P \wedge \neg B\}}$$

➤ Función cota o limitadora de un bucle:

- ✓ Las dos condiciones anteriores no garantizan que el bucle termine. Por tanto, además, habrá que asegurarse de la terminación del bucle. Para ello, se tiene que encontrar una expresión del número de iteraciones que realiza el bucle, en función de algún estado de cómputo, y demostrar que:
 1. Esa expresión se evalúa a un número entero positivo y, por tanto, tiene una cota inferior.
 2. El valor de la expresión decrece estrictamente en cada iteración.
- ✓ La búsqueda de esa expresión se basa en el concepto de **función limitadora**. Una función limitadora es una función que a cada estado del algoritmo da un número entero ($t: estado \rightarrow Z$) y que satisface:
 - Que se mantiene no negativa en cada iteración: 1. $P \wedge B \Rightarrow t \geq 0$
 - Decrece en cada iteración: 2. $\{P \wedge B \wedge t = T\} S \{t < T\}$

➤ **Función cota o limitadora de un bucle (continuación...):**

- ✓ Teniendo en cuenta lo anterior, la regla general para un bucle **mientras** es:

$$\frac{\{P \wedge B\}S\{P\}, P \Rightarrow \text{Dom}(B), P \wedge B \Rightarrow t \geq 0, \{P \wedge B \wedge t = T\}S\{t < T\}}{\{P\}\text{Mientras } B \text{ hacer } S \text{ fmientras } \{P \wedge \neg B\}}$$

- ✓ Para construir una **función limitadora** basta con observar las variables que son modificadas por el cuerpo S del bucle, y construir con (alguna de) ellas una expresión entera t que de una indicación del número de iteraciones que quedan por realizar en una determinada iteración y que, por tanto:
 - Decrezca en cada iteración.
 - Garantice que es no negativa siempre que se cumpla $P \wedge B$.
- ✓ **Ejemplo:** Obtener la función limitadora del algoritmo de la transparencia 22.

➤ Derivación del invariante a partir de la postcondición

- ✓ Obedece más a una heurística que a un método.
- ✓ La idea de partida es que el invariante es un predicado cuya componente principal es un predicado más débil que la postcondición, por lo que debilitando ésta se puede llegar a él.
- ✓ Se basa en las dos propiedades siguientes:
 - ✓ En el momento en que el bucle termina, se satisface P junto con $\neg B$, pero también y R . Es decir, $P \wedge \neg B \Rightarrow R$.
 - ✓ Sin embargo, mientras el bucle sigue iterando, se satisface P pero no necesariamente R . Es decir, sólo alguno de los estados de P (los que cumplen $\neg B$) son también estados de R .
- ✓ Entonces, debilitando R se puede llegar a P .

➤ Derivación del invariante a partir de la postcondición (continuación...)

- ✓ Las técnicas para debilitar un predicado son:
- ✓ Si R está en forma conjuntiva ($R_1 \wedge R_2$):
 - ✓ Se toma P como uno de los dos operandos (en el que sea más fácil dar valores a sus variables libres que hagan que se cumpla). El otro sería directamente $\neg B$ (que proporciona además B).
- ✓ Si R no está en forma conjuntiva:
 - ✓ Se construye R_1 sustituyendo cierta expresión $\Phi(x)$ de R (normalmente una constante) por una variable nueva w , y R_2 como la ecuación de sustitución $w = \Phi(x)$. De esta forma, se debe cumplir:

$$R_1(x, w) \wedge (w = \Phi(x)) \Rightarrow R$$
 - ✓ Tomándose $R_1 \wedge \text{Dom}(w)$ como P y $w = \Phi(x)$ como $\neg B$.

➤ Derivación del invariante a partir de la postcondición (continuación...)

✓ Resumen calculo del invariante:

1. Si la postcondicion del bucle es una conjunción de dos operandos, decidir cual de los dos operandos es la propuesta de **P** y cual la condición de salida del bucle $\neg \mathbf{B}^{(*)}$.
2. En caso contrario, aplicar una ecuación de sustitución para ponerla de esa forma, para prodecer como en el punto 1.

** Si se esta obteniendo un invariante para verificar un algoritmo, la elección del conjuntante del punto 1 o la ecuación de sustitución del punto 2 viene impuesta por la condición del bucle.*

3. En el caso de que la propuesta de invariante no sirva, por ser aún demasiado fuerte, se debilitará aún mas haciendo más sustituciones.

➤ Derivación del invariante a partir de la postcondición (continuación...)

1. Determinar el invariante a partir de la postcondición del algoritmo que suma los n primeros números naturales.
2. Determinar el invariante a partir de la postcondición del algoritmo que dado un vector $\mathbf{a}$ de n booleanos, comprueba si el valor de cada componente implica el valor de la siguiente
3. Determinar el invariante a partir de la postcondición del algoritmo que dado un vector de enteros $\mathbf{a}$ definido de 0 a n representando los coeficientes de un polinomio de grado n , compruebe si otro vector $\mathbf{b}$ definido de 0 a $n-1$ representa su derivada.
4. Determinar el invariante a partir de la postcondición del algoritmo que dado un vector $\mathbf{a}$ de n enteros, compruebe si la suma de sus elementos es 0 y todas las sumas parciales del primero hasta cada uno de sus elementos no son negativas.

Índice: Tema 2

2.1 El sistema formal de Hoare

2.1.1 Introducción

2.1.2 Reglas generales

2.1.3 Reglas de las instrucciones del lenguaje

2.1.4 Instrucciones iterativas

2.2 Derivación formal de algoritmos iterativos

2.2.1 Introducción

2.2.2 Derivación de instrucciones de asignación

2.2.3 Derivación de bucles

2.3 Verificación formal de algoritmos iterativos

2.3.1 Proceso de verificación

- Las ideas básicas a seguir para derivar formalmente un algoritmo iterativo, partiendo de su especificación formal son las siguientes:
 1. El punto de partida es la postcondición.
 2. La precondition juega un papel secundario.
 3. Se utilizan asertos intermedios para facilitar el proceso.
 4. La derivación del algoritmo y su verificación formal se desarrollan en paralelo.
 5. La corrección es la que dirige el proceso.
- Vamos a separar el estudio de las instrucciones a derivar en dos casos:
 - ✓ *Derivación de instrucciones de asignación.*
 - ✓ *Derivación de bucles.*

- Con estos dos casos, el objetivo final es ser capaz de derivar un algoritmo iterativo con la siguiente estructura:

```
{Q}
  inic;
  mientras B hacer {P}
    S
  Fmientras
{R}
```

- Además, el invariante ***P*** debe:
 - ✓ Ser lo suficientemente débil como para que se cumpla antes y después de cada iteración.
 - ✓ Pero lo suficientemente fuerte para que de su conjunción y la condición de salida del bucle se esté cerca de cumplir la postcondición.

Índice: Tema 2

2.1 El sistema formal de Hoare

2.1.1 Introducción

2.1.2 Reglas generales

2.1.3 Reglas de las instrucciones del lenguaje

2.1.4 Instrucciones iterativas

2.2 Derivación formal de algoritmos iterativos

2.2.1 Introducción

2.2.2 Derivación de instrucciones de asignación

2.2.3 Derivación de bucles

2.3 Verificación formal de algoritmos iterativos

2.3.1 Proceso de verificación

- Para poder derivar una instrucción, por simple o compleja que sea, se tienen que conocer los asertos que se deben cumplir antes y después de la ejecución de dicha instrucción.
- Partiendo de este conocimiento, si I es una instrucción que debe hacer correcta la terna $\{Q\} I \{R\}$, se sabe que I debe hacer correcta $Q \Rightarrow pmd(I, R)$
- Se tienen dos casos:
 - ✓ **Derivación de instrucciones simples:** Si R tiene una expresión de asignación $x:=E$, se prueba con ella para demostrar que $Q \Rightarrow pmd(x := E, R)$, en cuyo caso esa es la instrucción que se necesita.
 - ✓ **Derivación de instrucciones complejas:** Cuando, partiendo del mismo planteamiento del punto anterior, la propuesta de I no sirve, se insertan asignaciones $I \wedge E_x$ adicionales que verifiquen $Q \Rightarrow pmd(I \wedge E_x, R)$.

➤ *Derivar las siguientes instrucciones de asignación.*

1. Instrucción 1:

$$\{Q \equiv x = X \wedge y = Y\}$$

I

$$\{R \equiv x = Y \wedge y = X\}$$

2. Instrucción 2:

$$\{Q \equiv x = X \wedge y = Y \wedge AB = u + xy\}$$

I

$$\{R \equiv x = X \mathit{div} 2 \wedge y = 2Y \wedge AB = u + xy\}$$

Índice: Tema 2

2.1 El sistema formal de Hoare

2.1.1 Introducción

2.1.2 Reglas generales

2.1.3 Reglas de las instrucciones del lenguaje

2.1.4 Instrucciones iterativas

2.2 Derivación formal de algoritmos iterativos

2.2.1 Introducción

2.2.2 Derivación de instrucciones de asignación

2.2.3 Derivación de bucles

2.3 Verificación formal de algoritmos iterativos

2.3.1 Proceso de verificación

➤ Esquema del bucle

- ✓ El esquema de bucle que se va a utilizar para su derivación es el siguiente:

```

{Q}
Inic
    ← 1
mientras B hacer{P}
    ← 2
Restablecer
{T}
Avanzar
    ← 3
fmientras
    ← 4
{R}
    
```

- ✓ Donde para no repetir la escritura del invariante **P**, en los puntos donde se debe cumplir (1,2,3 y 4), éste se escribirá una sola vez a continuación de la palabra clave **hacer**. Además ,en 2 se debe cumplir junto con **B** (**P** $\wedge$ **B**) y en 4 con $\neg$ **B** (**P** $\wedge$ $\neg$ **B**).

➤ Esquema del bucle (continuación...)

- ✓ **Inic** son las instrucciones anteriores al bucle, que suelen ser asignaciones simples de las variables libres del invariante que no sean parámetros de entrada y que hagan que el invariante se cumpla.
- ✓ El cuerpo del bucle está constituido por las instrucciones denotadas por:
 - ✓ **Avanzar**, que representa a las instrucciones que hacen que las variables que intervienen en la expresión **B** avancen hacia la condición de salida del bucle, es decir, hacia $\neg B$.
 - ✓ **Restablecer**, que representa a las instrucciones que modifican las variables libres del invariante no modificadas por **Avanzar**, y que se incluyen para restablecer la invarianza de **P** normalmente destruida por **Avanzar**.
 - ✓ **T** es un **aserto intermedio**, que habrá que calcular una vez concretada **Avanzar**, cuya utilidad es facilitar el cálculo de **Restablecer**.

➤ Proceso de derivación

- ✓ La derivación de un bucle sigue unos pasos muy definidos:
 1. Obtener el invariante ***P*** a partir de la postcondición ***R***. Recordar que debe ser correcta $P \wedge \neg B \Rightarrow R$, por lo que en este proceso también se obtiene $\neg B$.
 2. Obtener la condición del bucle ***B*** a partir de $\neg B$.
 3. Obtener las instrucciones de ***Inic***. Para ello, tener en cuenta que se debe satisfacer $\{Q\} \text{Inic} \{P\}$.
 4. Obtener las instrucciones de ***Avanzar***. Para ello, partiendo del valor asignado a las variables en ***Inic***, ver de qué forma hay que modificar las que aparecen en ***B*** para hacer que vayan hacia unos valores que hagan que se cumpla $\neg B$.
 5. Obtener las instrucciones de ***Restablecer***. Para ello, se introduce un aserto intermedio ***T*** que debe satisfacer $\{P \wedge B\} \text{Restablecer} \{T\} \text{Avanzar} \{P\}$.

➤ Proceso de derivación (continuación...)

5. Para obtener las instrucciones de **Restablecer**:

1. Si $T \equiv \text{pmd}(\text{Avanzar}, P)$ hace correcta la deducción $P \wedge B \Rightarrow T$, entonces **Restablecer** $\equiv \text{nada}$.
2. En caso contrario, se calcula **Restablecer** mediante modificaciones de las variables libres del invariante no modificadas por **Avanzar**, de tal forma que se haga correcta la deducción $P \wedge B \Rightarrow \text{pmd}(\text{Restablecer}, T)$.

✓ Derivación de **bucles en secuencia y anidados**:

- ✓ Si se necesitan varios bucles, se deriva cada uno por separado.
- ✓ Si se necesitan bucles anidados, se deriva cada uno por separado empezando desde el más externo hacia el más interno.

➤ *Derivar el algoritmo con el que calcular la suma de los **n** primeros números naturales.*

✓ La especificación es:

$$\{Q \equiv n \geq 0\}$$

fun suma(*n: ent*) ***dev***(*s: ent*)

$$\{R \equiv s = \sum \alpha \in \{0..n\}. \alpha\}$$

✓ Y su invariante:

$$\{P \equiv (s = \sum \alpha \in \{0..i\}. \alpha) \wedge (0 \leq i \leq n)\}$$

Índice: Tema 2

2.1 El sistema formal de Hoare

2.1.1 Introducción

2.1.2 Reglas generales

2.1.3 Reglas de las instrucciones del lenguaje

2.1.4 Instrucciones iterativas

2.2 Derivación formal de algoritmos iterativos

2.2.1 Introducción

2.2.2 Derivación de instrucciones de asignación

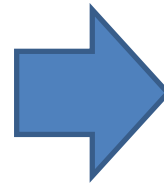
2.2.3 Derivación de bucles

2.3 Verificación formal de algoritmos iterativos

2.3.1 Proceso de verificación

➤ Esquema del bucle

- La verificación formal de un algoritmo iterativo se basa en la aplicación de las reglas vistas anteriormente.
- La estrategia de la verificación se basa en dividir el algoritmo en partes y verificar por separado cada una de ellas (aplicación de la regla de composición de instrucciones).
- Para ilustrar la verificación de algoritmos iterativos, ésta se centrará en algoritmos que sigan el siguiente esquema, ya que es suficientemente representativo:



```

{Q}
Inic
mientras B hacer {P}
  Restablecer
  {T}
  Avanzar
fmientras
{R'}
Fin
{R}
  
```

- **Esquema del bucle (continuación...)**
- Si el algoritmo fuese más complejo:
 - ✓ Si tuviese varios bucles en secuencia, se utilizarían predicados intermedios y se verificarán por separado.
 - ✓ Si tuviese bucles anidados, se verificaría el más interior y se trataría como una caja negra en el bucle externo superior.
- Un aspecto crucial para la verificación es encontrar el invariante del bucle, si éste no viene ya dado. Las ideas básicas que guían esta búsqueda son las siguientes:
 - ✓ Debe ser suficientemente fuerte para que junto con $\neg B$ lleve a R' .
 - ✓ Debe ser suficientemente débil para que *Inic* haga que se cumpla y después de la ejecución de *S* se mantenga la invarianza.

➤ Proceso de verificación

- ✓ La verificación de un bucle también sigue unos pasos muy definidos:
 1. Obtención de la postcondición del bucle R' :
 - Si **Fin** no tiene contenido entonces $R' \equiv R$.
 - En caso contrario $R' \equiv pmd(Fin, R)$.
 2. Proponer el invariante P y la función de cota (limitadora) del bucle t . Para ello se utilizan las mismas heurísticas presentadas en derivación.
 3. **Satisfacción de la postcondición del bucle R'** . Para ello, se debe comprobar que cuando se sale del bucle se llega a un estado que la satisface, es decir, $P \wedge \neg B \Rightarrow R'$.
 4. **Satisfacción inicial del invariante P** . Para ello, se debe comprobar el cumplimiento del invariante a la llegada al bucle, es decir, $\{Q\} \text{ Inic } \{P\}$.

➤ Proceso de verificación (continuación...)

5. **Mantenimiento de la invarianza en cada iteración.** Para ello, se debe comprobar de que después de cada iteración se mantiene la invarianza, es decir, $\{P \wedge B\} S \{P\}$.
 6. **Cota inferior positiva.** Para ello, se debe comprobar que la función cota tiene una cota inferior positiva, es decir $P \wedge B \Rightarrow t \geq 0$.
 7. **Decrecimiento del número de iteraciones.** Para ello, se debe comprobar que el bucle termina, para lo que es necesario que t decrezca en cada iteración, es decir, $\{(P \wedge B) \wedge (t = T)\} S \{t < T\}$.
- ✓ Aunque los pasos 3, 4, 5, 6 y 7 se pueden realizar en cualquier orden, conviene probar primero los pasos 3 y 4 (los límites del bucle).
 - ✓ Si falla la verificación de los pasos 3 y 4, suele ser porque el invariante no es el apropiado.

➤ Verificar el algoritmo con el que calcular la suma de los n primeros números naturales.

✓ Algoritmo:

$\{Q \equiv n \geq 0\}$

fun suma($n: \text{ent}$) dev($s: \text{ent}$)

$i := 0;$

$s := 0;$

mientras $i < n$ **hacer**

$s := s + i + 1;$

$i := i + 1$

fmientras

ffun

$\{R \equiv s = \sum \alpha \in \{0..n\}. \alpha\}$

➤ *Derivar y verificar los siguientes algoritmos:*

1. Un algoritmo que dado un vector $\mathbf{p}$ de $n+1$ naturales, representando los coeficientes de un polinomio de grado n construya otro vector $\mathbf{d}$ con los coeficientes de su derivada.
2. Dado un natural n , se desea una función iterativa completamente verificada que devuelva el número de los divisores de n , incluidos 1 y n , por ejemplo ***divisores***(12)=6.
3. En una empresa cada empleado debe fichar al entrar y al salir. Para cada trabajador se tiene la siguiente estructura, $\mathbf{h}: \text{nat}[1..N, 1..2]$, donde $\mathbf{h}[i, 1]$ y $\mathbf{h}[i, 2]$ representan la hora de entrada y salida, respectivamente, de un empleado el i -ésimo día del mes. Se pide diseñar un algoritmo iterativo para calcular el total de horas trabajadas por el empleado al mes.